

Laboratórios de Sistemas e Serviços

Caderno de Apoio à Teórica

Mário Antunes

14 de maio de 2026

Conteúdo

1	Foundations of Logic	1
1.1	Prelude: The Boolean Perspective	1
1.2	The Architecture of a Logic System	2
1.3	Propositional Logic	3
1.4	First-Order Logic (FOL)	3
1.5	Interpretations, Models, and Entailment	4
1.6	Rewrite Rules and Normal Forms	4
1.7	The Resolution Inference Rule	5
1.8	Horn Clauses and Logic Programming	7
2	Intelligent Agents	9
2.1	Defining the Agent	9
2.2	The Agent Interaction Loop	9
2.3	Characterizing the Environment	11
2.4	The Spectrum of Agent Architectures	12
2.5	Hierarchy of Agent Types	13
3	Logic-Based Agents	15
3.1	Architecture of a Knowledge-Based Agent	15
3.2	The Language of First-Order Logic (FOL)	16
3.3	Inference and Reasoning	17
3.4	Knowledge Representation Challenges	17
3.5	Case Study: The Wumpus World	18
3.6	Rule-Based Systems and Logic Programming	19
3.7	Summary	19
4	Prolog and Logic Programming	21
4.1	The Foundations of Prolog Syntax	21
4.2	The Mechanics of Inference	22
4.3	List Processing and Recursion	23
4.4	Controlling Search: The Cut Operator (!)	24
4.5	Dynamic Knowledge Bases and Agent Memory	24
4.6	Case Study: The Hot or Cold Agent	25
4.7	Integration: Prolog and Python	25

5	Rule-Based Systems and Rule Engines	27
5.1	From Logic to Automation	27
5.2	The Architecture of a Rule Engine	27
5.3	The Decision Cycle: Match-Resolve-Act	28
5.4	Real-World Applications	30
5.5	Modern Implementations: GoRules and JDM	30
5.6	Conclusion	31
6	Bayesian Agents and Probabilistic Reasoning	33
6.1	From Certainty to Uncertainty	33
6.2	The Heart of Bayesian Inference: Bayes' Rule	34
6.3	Bayesian Networks	34
6.4	Modeling Decisions over Time: Markov Processes and MDPs	35
6.5	Training and Implementation	36

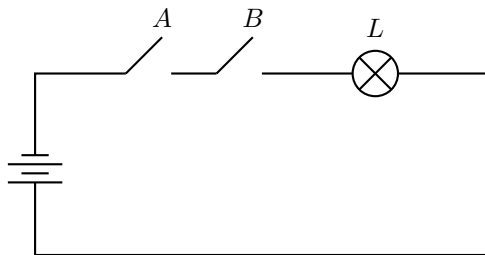
1 Foundations of Logic

Logic is the cornerstone of formal reasoning, providing the mathematical framework necessary to represent knowledge and derive new information from existing facts. At its core, logic allows us to abstract away from the physical world and operate within a symbolic system where truths can be calculated with precision. In the context of computer science and artificial intelligence, logic systems are the engines that power automated reasoning, rule-based systems, and formal verification.

1.1 Prelude: The Boolean Perspective

Before exploring complex symbolic systems, it is instructive to consider the most fundamental form of logic: Boolean logic. Named after George Boole, this system operates on two states—typically denoted as 1 (True, On, or High) and 0 (False, Off, or Low). Boolean logic is the foundation of digital circuitry, where physical switches represent logical variables.

Consider a simple electrical circuit involving two switches, A and B , connected in series to a lamp L . The lamp will only illuminate if both switches are closed. This physical arrangement is a direct implementation of the logical **AND** ($\wedge$) operation.



Mathematically, we represent the state of the lamp as a function of the switches:

$$L = A \wedge B$$

This relationship is fully described by a **truth table**, which enumerates every possible combination of inputs and the resulting output:

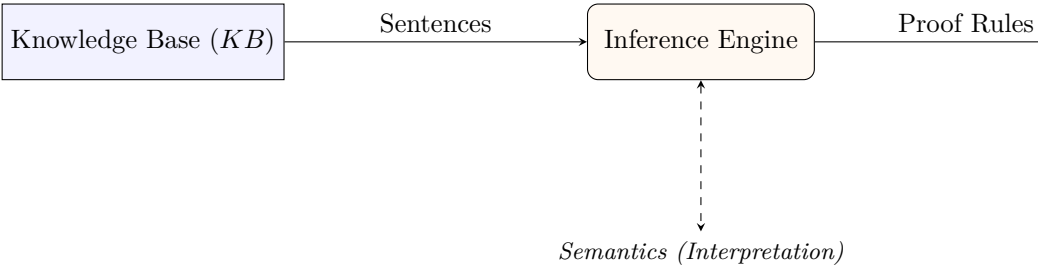
Switch <i>A</i>	Switch <i>B</i>	Lamp (<i>L</i>)
0	0	0
0	1	0
1	0	0
1	1	1

The power of this abstraction is that it allows us to predict the state of the lamp without physical observation. If we know that *A* is True and *B* is True, logic dictates that *L* must be True.

1.2 The Architecture of a Logic System

A formal logic system is not merely a set of symbols; it is a structured environment composed of three essential pillars:

- 1. **Syntax:** This defines the “grammar” of the system. It specifies the alphabet of symbols and the rules for combining them into **Well-Formed Formulas (WFF)**. Syntax determines what is *legal* to write, regardless of whether it is true or false.
- 2. **Semantics:** This provides the “meaning” of the formulas. It establishes a mapping between the symbolic expressions and the domain of discourse (the “real world”). In propositional logic, for instance, semantics are often defined using truth tables that assign truth values to variables.
- 3. **Inference Rules:** These are the algorithms or procedural rules that allow for the derivation of new formulas from existing ones. An inference rule is a transformation that preserves truth; if the starting formulas are true, the derived formulas must also be true.



1.3 Propositional Logic

Propositional logic is the simplest branch of formal logic. It deals with **propositions**, which are declarative statements that are either True or False. For example, “The sky is blue” is a proposition, whereas “Close the door!” is not.

In this system, we use **propositional variables** (such as P, Q, R) to represent these statements. These variables are then combined using **logical connectives**:

- **Negation** ($\neg$): “Not P ”. Flips the truth value.
- **Conjunction** ($\wedge$): “ P and Q ”. True only if both are true.
- **Disjunction** ($\vee$): “ P or Q ”. True if at least one is true.
- **Implication** ($\Rightarrow$): “If P , then Q ”. This is a causal or conditional link. It is only false if P is true and Q is false.
- **Biconditional** ($\Leftrightarrow$): “ P if and only if Q ”. True if both have the same truth value.

Example: Let P represent “It is raining” and Q represent “The ground is wet”. The formula $P \Rightarrow Q$ formalizes the rule that rain implies a wet ground. If we observe P (it is raining), we can infer Q (the ground is wet).

1.4 First-Order Logic (FOL)

While propositional logic is powerful, it is limited by its inability to describe individual objects or the relationships between them. It treats every statement as an indivisible unit. **First-Order Logic (FOL)**, also known as Predicate Logic, extends this by introducing a more granular vocabulary:

- **Objects (Constants)**: Represent specific entities in the world, such as Alice, Aveiro, or the number 5.
- **Functions**: Mappings that take one or more objects and return another object. For instance, $FatherOf(Bob)$ returns the object representing Bob’s father.
- **Predicates (Relations)**: Mappings that take objects and return a truth value. For example, $Greater(5,3)$ is True, while $Near(Lisbon, Oporto)$ might be False.
- **Variables**: Placeholders (like x, y) that can stand for any object in the domain.

1.4.1 Quantifiers

FOL introduces **quantifiers** to express statements about sets of objects:

1. **Universal Quantifier** ($\forall$): Indicates that a property holds for **all** objects in the domain.
 - *Example:* “All humans are mortal” $\rightarrow \forall x(Human(x) \Rightarrow Mortal(x))$.
 - *Note:* A common pitfall is using $\wedge$ with $\forall$. $\forall x(Human(x) \wedge Mortal(x))$ would mean “Everything in the universe is both human and mortal.”
2. **Existential Quantifier** ($\exists$): Indicates that there is **at least one** object in the domain for which the property holds.
 - *Example:* “Some bird cannot fly” $\rightarrow \exists x(Bird(x) \wedge \neg CanFly(x))$.
 - *Note:* Using $\Rightarrow$ with $\exists$ is usually a mistake. $\exists x(Bird(x) \Rightarrow \neg CanFly(x))$ is vacuously true if there exists anything in the universe that is *not* a bird.

1.5 Interpretations, Models, and Entailment

The relationship between syntax and the world is defined by an **Interpretation**. * In Propositional Logic, an interpretation is simply an assignment of True/False to every variable. * In FOL, it is a complex mapping of constant symbols to objects, predicate symbols to relations, and function symbols to functional mappings within a specific domain.

A **Model** is an interpretation in which a given formula evaluates to True. We say a formula is **Satisfiable** if it has at least one model. If a formula is True in *every* possible interpretation, it is a **Tautology** or is **Valid**.

Entailment ($\models$) is the fundamental relationship in logic. We say that a Knowledge Base (KB) entails a sentence α ($KB \models \alpha$) if and only if in every model where KB is true, α is also true. This is the goal of reasoning: to find what else must be true given what we already know.

1.6 Rewrite Rules and Normal Forms

Logic allows for algebraic manipulation through **Equivalences**. These are rules that allow us to transform formulas while preserving their truth value.

1.6.1 Key Equivalences

- **De Morgan's Laws:**

$$\neg(A \wedge B) \equiv \neg A \vee \neg B$$

$$\neg(A \vee B) \equiv \neg A \wedge \neg B$$

- **Implication Elimination:**

$$(A \Rightarrow B) \equiv (\neg A \vee B)$$

- **Quantifier Duality:**

$$\neg \forall x P(x) \equiv \exists x \neg P(x)$$

$$\neg \exists x P(x) \equiv \forall x \neg P(x)$$

1.6.2 Conjunctive Normal Form (CNF)

For automated reasoning, it is often necessary to convert formulas into a standardized format called **Conjunctive Normal Form (CNF)**. A formula in CNF is a conjunction ($\wedge$) of one or more **clauses**, where each clause is a disjunction ($\vee$) of **literals** (atoms or negated atoms).

Example: The formula $(A \vee B) \wedge (\neg B \vee C \vee D) \wedge (\neg A)$ is in CNF. This format is essential for the Resolution algorithm.

1.7 The Resolution Inference Rule

Resolution is a powerful and computationally efficient inference rule that serves as the backbone of many automated theorem provers. It operates on clauses in CNF and generalizes the classical *Modus Ponens*.

1.7.1 Propositional Resolution

The rule states that if we have two clauses where one contains a literal P and the other contains its negation $\neg P$, we can “resolve” them to produce a new clause containing all other literals from both:

$$\frac{A \vee B, \quad \neg B \vee C}{A \vee C}$$

Logic: If B is false, then A must be true (from the first clause). If B is true, then C must be true (from the second clause). Since B must be either true or false, it follows that $A \vee C$ must be true.

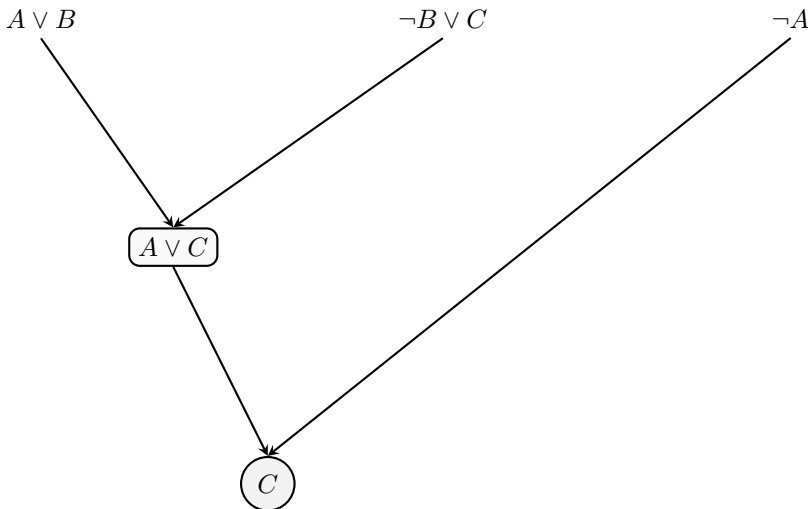
1.7.2 Unification

In FOL, literals often contain variables. To resolve them, we must first make them identical through **Unification**. This involves finding a **substitution** θ (a mapping of variables to terms) that makes two expressions match.

Example: To unify $Knows(John, x)$ and $Knows(y, Mother(y))$, we apply the substitution $\theta = \{y/John, x/Mother(John)\}$, resulting in the unified atom $Knows(John, Mother(John))$.

1.7.3 Proof by Refutation

Resolution is typically used in a **Proof by Refutation** (*Reductio ad Absurdum*). To prove that $KB \models \alpha$, we: 1. Assume the negation of the goal: $\neg\alpha$. 2. Add $\neg\alpha$ to the KB . 3. Convert the entire set to CNF. 4. Repeatedly apply Resolution. 5. If we derive the **Empty Clause** ($\square$), which represents a contradiction (*False*), then the original goal α must be a logical consequence of KB .



1.8 Horn Clauses and Logic Programming

In many practical applications, we restrict the logic to a subset called **Horn Clauses**. A Horn Clause is a clause with **at most one positive literal**. This restriction allows for much faster inference, which is the basis for the **Prolog** programming language.

Horn clauses come in three flavors: 1. **Rules**: $\neg P_1 \vee \dots \vee \neg P_n \vee H$, equivalent to $(P_1 \wedge \dots \wedge P_n) \Rightarrow H$. In Prolog: $H \text{ :- } P_1, \dots, P_n$. 2. **Facts**: A single positive literal H . In Prolog: H . 3. **Goals**: A clause with no positive literals $\neg G_1 \vee \dots \vee \neg G_n$. In Prolog: $\text{:- } G_1, \dots, G_n$.

Systems using Horn clauses can employ **Forward Chaining** (starting from facts to reach a conclusion) or **Backward Chaining** (starting from a goal and working back to facts), making them highly efficient for expert systems and automated reasoning.

2 Intelligent Agents

In the study of artificial intelligence, the concept of an **agent** serves as a unifying framework for designing systems that can perceive their surroundings and take actions to achieve specific objectives. Rather than viewing AI as a collection of isolated algorithms, the agent-oriented perspective treats the system as a situated entity—a “being” that exists in a continuous loop of interaction with its environment.

2.1 Defining the Agent

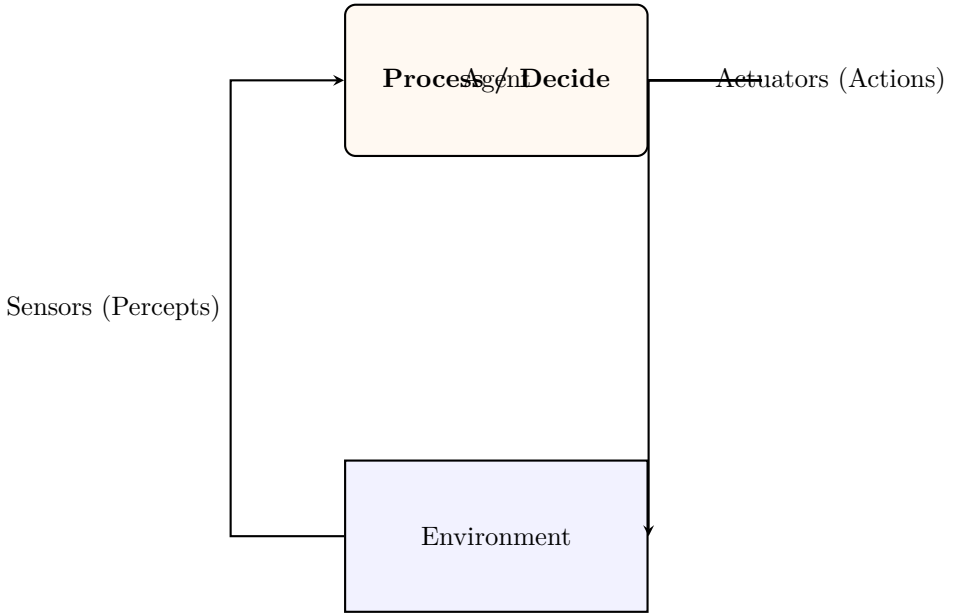
At its most fundamental level, an agent is anything that can be viewed as perceiving its environment through **sensors** and acting upon that environment through **actuators**. This definition is intentionally broad, encompassing a vast spectrum of entities:

- **Biological Agents:** Humans perceive the world through eyes, ears, and touch, and act using muscles and vocal cords.
- **Robotic Agents:** A vacuum cleaner uses bumpers and infrared sensors to detect walls and motors to navigate the floor.
- **Software Agents:** A web crawler “senses” the internet by reading HTML files and “acts” by following links or indexing data.

The defining characteristic of an agent is its degree of autonomy—the ability to exercise control over its own actions to achieve goals without constant human intervention.

2.2 The Agent Interaction Loop

The life of an agent is defined by a cyclic process often referred to as the **Sense-Think-Act** loop. This interaction establishes a dynamic relationship where the agent is both influenced by and an influencer of its environment.



The loop consists of four distinct phases: 1. **Sense**: The agent gathers data from the environment via its sensors. 2. **Process/Decide**: The internal “intelligence” of the agent processes the sensory input to determine the best course of action. 3. **Act**: The agent executes the chosen action through its actuators. 4. **Alter**: The action modifies the state of the environment, which is then sensed in the next iteration of the loop.

Example: Consider a pilot flying an aircraft. The pilot **senses** the altitude via the altimeter, **decides** that the plane is too low, **acts** by pulling back on the yoke, and thereby **alters** the plane’s pitch and altitude, which will be sensed again in the next moment.

2.2.1 Mathematical Formalization

To study agents rigorously, we define the **Agent Function** (f), which maps the entire history of percepts (the **percept sequence**) to a specific action:

$$f : P^* \rightarrow A$$

where P^* represents the set of all possible sequences of percepts, and A is the set of actions. In practice, the agent function is implemented by an **Agent Program** running on physical architecture.

2.3 Characterizing the Environment

The “World” in which an agent operates—the **Environment**—is as critical to the design of the agent as the agent’s own code. The environment defines the physics, the constraints, and the rules of the game. We describe the state of the environment at any time t as S_t .

The evolution of the environment can be modeled by a **Transition Function** (T):

$$S_{t+1} = T(S_t, A_t)$$

This indicates that the next state is a result of the current state and the agent’s action. However, agents rarely have perfect information. This leads us to the **Dimensions of the Environment**, which classify the challenges an agent might face:

1. **Observability:** In a **Fully Observable** environment (like Chess), the agent’s sensors detect the entire state. In a **Partially Observable** environment (like Poker), the agent must deal with uncertainty and “hidden” information.
2. **Determinism:** In a **Deterministic** environment, the next state is perfectly predictable. In a **Stochastic** environment, actions have probabilistic outcomes (e.g., throwing dice).
3. **Dynamics:** A **Static** environment remains unchanged while the agent is “thinking.” A **Dynamic** environment (like driving) continues to evolve, forcing the agent to act quickly.

2.3.1 Discrete vs. Continuous Spaces

One of the most significant distinctions in AI design is whether the world and the actions are discrete or continuous.

- **Discrete World:** The world consists of finite, distinct states (e.g., squares on a grid). Transitions happen in “turns” or steps. Math involves graph theory and state machines.
- **Continuous World:** States are defined by real numbers (e.g., coordinates (x, y, z) in 3D space). Time flows smoothly. Math involves calculus and physics engines.

Taxonomy of Environments:

	Discrete Action	Continuous Action
Discrete World	Chess: Fixed squares and specific moves.	Ant Colony: Fixed nodes, but pheromone intensity is a real number.
Continuous World	Platformer Games: Smooth movement, but jump/fire buttons are binary.	Robotics: Physics-based world and motor torques range over a scale.

2.4 The Spectrum of Agent Architectures

An agent’s “intelligence” resides in how it translates percepts into actions. Different paradigms of AI offer different ways to implement this “Process/Decide” step.

2.4.1 Logic-Based Agents

These agents use formal logic (such as Propositional or First-Order Logic) to represent knowledge. They maintain a **Knowledge Base (KB)** of facts and rules. * **Process:** Sense $\rightarrow$ Logical Fact $\rightarrow$ Inference Engine $\rightarrow$ Action. * **Example:** A tax-calculating agent that follows a strict set of legal rules to determine a refund amount.

2.4.2 Planning Agents

These agents use a model of the world to “look ahead” into the future. They search for a sequence of actions that will lead to a desired goal. * **Process:** Search algorithms (like A) *find a path through a state-space graph*. **Example:** A warehouse robot planning a path to a specific shelf while avoiding obstacles.

2.4.3 Learning Agents (Reinforcement Learning)

Rather than being programmed with rules, these agents learn through trial and error. They receive a **Reward (R)** signal from the environment and update their **Policy (π)** to maximize their cumulative “happiness.” * **Process:**

Optimization of a value function or policy. * **Equation (Value Function):** $V(s) = E[\sum_{t=0}^{\infty} \gamma^t R_t | S_0 = s]$ * **Example:** A dog learning that “sitting” when commanded leads to a treat.

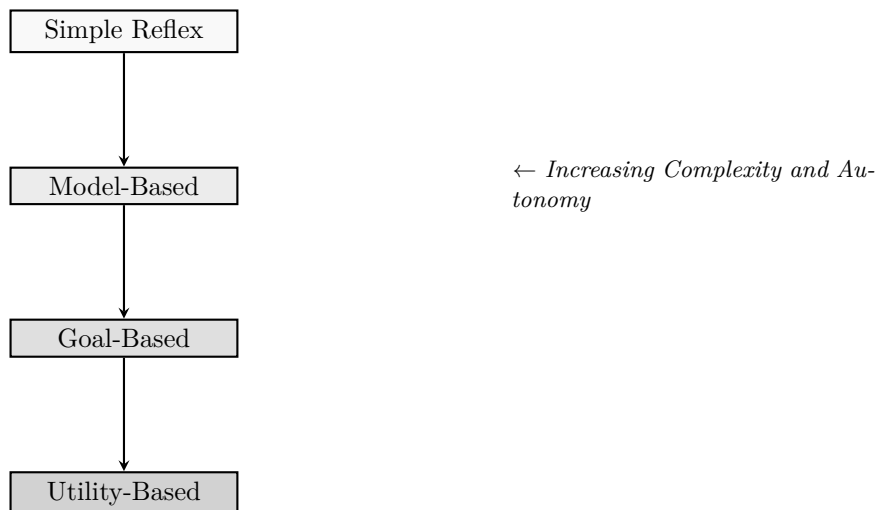
2.4.4 Agentic AI (LLMs)

The newest frontier involves using Large Language Models as the reasoning engine. These agents can handle natural language, use tools, and perform complex reasoning through patterns like **ReAct** (Reason + Act). * **Example:** A coding assistant that analyzes a bug, searches the file system, and applies a fix autonomously.

2.5 Hierarchy of Agent Types

As agents become more sophisticated, they incorporate more complex models of the world and their own objectives.

1. **Simple Reflex Agents:** Act based solely on the current percept, ignoring history.
 - *Rule:* if condition then action
 - *Example:* A simple thermostat turning on the heater when the temperature drops.
2. **Model-Based Reflex Agents:** Maintain an internal state that tracks aspects of the world that are currently unobservable.
 - *Mechanism:* They use a “World Model” to predict how the environment evolves.
3. **Goal-Based Agents:** Act to achieve a specific target state. They can reason about the consequences of their actions in the future.
4. **Utility-Based Agents:** Use a continuous **Utility Function** to measure the “desirability” of a state. They choose actions that maximize *expected* utility, allowing them to make trade-offs (e.g., speed vs. safety in a taxi).



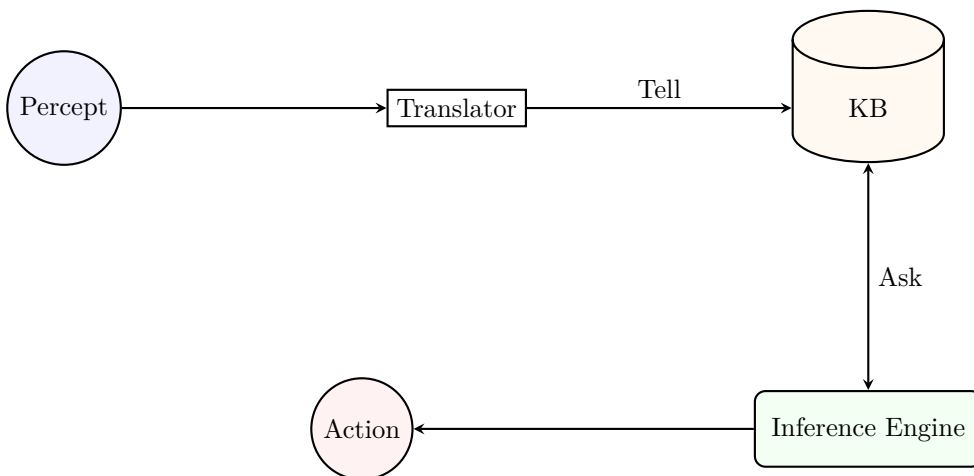
In conclusion, an intelligent agent is more than just a piece of code; it is a system designed to thrive in a specific environment. Whether through strict logic, exhaustive planning, or deep learning, the goal remains the same: to turn perception into purposeful action.

3 Logic-Based Agents

Logic-based agents represent a paradigm of artificial intelligence where reasoning is performed through the manipulation of symbolic representations. Unlike imperative systems that follow a fixed sequence of instructions, a logic-based agent operates declaratively: it maintains a **Knowledge Base (KB)** of facts and rules about the world and uses an **Inference Engine** to deduce what actions to take. This approach separates the “what” (the knowledge) from the “how” (the reasoning process), providing a transparent and mathematically rigorous framework for intelligent behavior.

3.1 Architecture of a Knowledge-Based Agent

A knowledge-based agent is structured around its ability to internalize perceptions as logical sentences and query its knowledge for optimal responses. The fundamental loop of such an agent involves receiving a percept, translating it into the internal logic language, “telling” the KB about this new information, and then “asking” the KB for the best action to execute.



The **Agent Function** is the abstract mapping from percept sequences to actions, $f : P^* \rightarrow A$. In a logic-based agent, this function is realized by the **Agent Program**, which implements the inference engine. This separation allows for **Declarative Programming**, where the developer describes the properties of the solution rather than the steps to reach it.

Example: In an imperative system, one might write `if wall_ahead: turn_left()`. In a declarative logic system, one defines the rule `Clear(ahead) \wedge \neg Wall(ahead) \implies CanMove(ahead)`. The agent then searches for a state where `CanMove(ahead)` is true to decide its next step.

3.2 The Language of First-Order Logic (FOL)

To represent complex worlds, we require the expressive power of **First-Order Logic (FOL)**. While Propositional Logic deals with simple facts, FOL introduces objects, relations (predicates), and functions.

3.2.1 Formal Syntax

The syntax of FOL is defined by a set of recursive rules, often represented in Backus-Naur Form (BNF). A formula can be an atomic formula, a combination of formulas using connectives ($\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$), or a quantified formula using $\forall$ (Universal) or $\exists$ (Existential).

$$\text{Term} \rightarrow \text{Function}(\text{Term}, \dots) \mid \text{Constant} \mid \text{Variable}$$

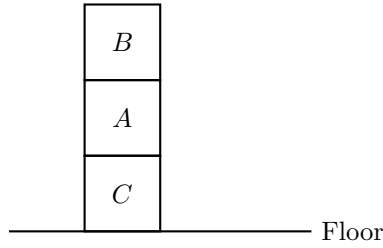
$$\text{AtomicSentence} \rightarrow \text{Predicate}(\text{Term}, \dots) \mid \text{Term} = \text{Term}$$

$$\text{Sentence} \rightarrow \text{AtomicSentence} \mid (\text{Sentence} \text{ Connective } \text{Sentence}) \mid \text{Quantifier Variable Sentence}$$

3.2.2 Semantics: Interpretations and Models

The meaning of a formula is determined by an **Interpretation**, which maps symbols to entities in a domain (the “Universe”). A **Model** is an interpretation that satisfies a formula (makes it true).

Example: The Blocks World Consider a domain $\{A, B, C, \text{Floor}\}$. We define the predicate $\text{OnTop}(x, y)$. An interpretation might be: $\text{OnTop}(B, A) = \text{True}$, $\text{OnTop}(A, C) = \text{True}$, and $\text{OnTop}(C, \text{Floor}) = \text{True}$. This model represents a stack of three blocks (B on A on C) on the floor.



3.3 Inference and Reasoning

Inference is the process of deriving new sentences from old ones. The most common rule is **Modus Ponens**:

$$\frac{P \Rightarrow Q, \quad P}{Q}$$

This rule is sound; if the premises are true, the conclusion is guaranteed to be true.

3.3.1 Unification

In FOL, we often need to match a general rule to a specific fact. **Unification** is the process of finding a substitution θ that makes two logical expressions identical. * **Example**: Unifying $At(Agent, x)$ with $At(Agent, Kitchen)$ yields the substitution $\theta = \{x/Kitchen\}$.

3.3.2 Resolution and Proof by Refutation

Resolution is a powerful inference rule that operates on clauses in Conjunctive Normal Form (CNF). A common technique is **Proof by Refutation**: to prove a goal G , we add $\neg G$ to the KB and attempt to derive a contradiction (the empty clause $\square$).

3.4 Knowledge Representation Challenges

Representing a changing world introduces several technical hurdles, most notably the **Frame Problem**.

3.4.1 The Frame Problem

When an agent performs an action, most facts about the world remain unchanged. However, in pure logic, we must explicitly state what *doesn't* change. If an agent moves a cup, the color of the walls remains the same, the temperature of the room remains the same, and the location of the floor remains the same. Listing every non-change for every action leads to a “combinatorial explosion” of axioms.

The solution is the use of **Successor-State Axioms**, which define the value of a property (a **Fluent**) at time $t + 1$ based on the actions at time t :

$$F(t + 1) \iff [\text{Action made } F \text{ true}] \vee [F(t) \wedge \neg(\text{Action made } F \text{ false})]$$

3.5 Case Study: The Wumpus World

The Wumpus World is a classic environment for testing logic-based agents. It consists of a 4×4 grid containing pits, a gold treasure, and a monster (the Wumpus).

- **Percepts:** The agent perceives a *Breeze* if adjacent to a pit, a *Stench* if adjacent to the Wumpus, and a *Glitter* if in the same square as the gold.
- **Rules:**

$$\forall x, y \text{ Breeze}(x, y) \Leftrightarrow \exists a, b \text{ Adj}(x, y, a, b) \wedge \text{Pit}(a, b)$$

$$\forall x, y \text{ Stench}(x, y) \Leftrightarrow \exists a, b \text{ Adj}(x, y, a, b) \wedge \text{Wumpus}(a, b)$$

By observing a breeze in (1,1) and nothing in (2,1), the agent can use resolution to deduce which adjacent cells are safe.

	Pit		
		Stench Area	
		Wumpus	
Breeze			
Agent			Gold

3.6 Rule-Based Systems and Logic Programming

Logic-based agents often employ specific reasoning strategies: * **Forward Chaining**: Data-driven reasoning. It starts with known facts and applies rules to see what else can be deduced. This is useful for real-time monitoring. * **Backward Chaining**: Goal-driven reasoning. It starts with a goal and works backward through rules to find supporting facts. This is the strategy used by **Prolog**.

3.6.1 Horn Clauses

To ensure efficient inference, many systems restrict themselves to **Horn Clauses**, which have at most one positive literal.

$$\neg P_1 \vee \neg P_2 \vee \dots \vee \neg P_n \vee Q \equiv (P_1 \wedge P_2 \wedge \dots \wedge P_n) \Rightarrow Q$$

In Prolog, this is written as $Q :- P_1, P_2, \dots, P_n$. This restriction allows the inference engine to use **Depth-First Search (DFS)** with backtracking to explore potential proofs efficiently.

3.7 Summary

Logic-based agents provide a rigorous foundation for AI. They are **symbolic**, meaning their knowledge is represented in human-readable forms; **transparent**, allowing the reasoning process to be inspected; and **rigorous**, providing mathematical guarantees about the correctness of their deductions. While they can be brittle in the face of uncertainty, they remain the backbone of expert systems, formal verification, and automated theorem proving.

4 Prolog and Logic Programming

Prolog, short for *Programmation en Logique* (Programming in Logic), is the premier language of the logic programming paradigm. Unlike imperative languages that require a programmer to specify a sequence of steps to solve a problem, Prolog allows the developer to describe the problem itself as a set of facts and rules. The underlying engine then uses automated reasoning—specifically **Resolution** and **Unification**—to find answers to queries. This chapter explores the syntax, mechanics, and practical applications of Prolog in the design of intelligent agents.

4.1 The Foundations of Prolog Syntax

Prolog programs are composed of three basic building blocks: **atoms**, **variables**, and **compound terms**. These elements are used to construct **facts** and **rules**, which together form the knowledge base.

- **Atoms:** These are constants that represent specific objects or properties. They must begin with a lowercase letter (e.g., `apple`, `hall`) or be enclosed in single quotes if they contain spaces or start with uppercase (e.g., `'John Smith'`).
- **Variables:** Variables are placeholders for terms and must begin with an uppercase letter or an underscore (e.g., `X`, `Result`, `_hidden`). The anonymous variable `_` is used when the value of a term is irrelevant.
- **Facts:** A fact is a simple assertion that a relationship holds. For example, `parent(john, mary).` states that John is the parent of Mary.
- **Rules:** A rule defines a relationship based on other relationships. It uses the “neck” operator `:-`, which represents logical implication ($\Leftarrow$). The general form is `Head :- Body.`, meaning “Head is true if Body is true.”

Example: The Family Tree

```
parent(john, mary).  
parent(mary, alice).  
grandparent(X, Y) :- parent(X, Z), parent(Z, Y).
```

In this snippet, `grandparent(X, Y)` is true if there exists some `Z` such that `X` is the parent of `Z` AND `Z` is the parent of `Y`.

4.2 The Mechanics of Inference

When a user issues a **Query** (using the `?-` operator), Prolog attempts to prove the query by searching its knowledge base. This process involves two core mechanisms: **Unification** and **Backtracking**.

4.2.1 Unification

Unification is the process of matching two terms by finding a **substitution** of variables that makes them identical. In Prolog, the `=` operator represents unification, not assignment.

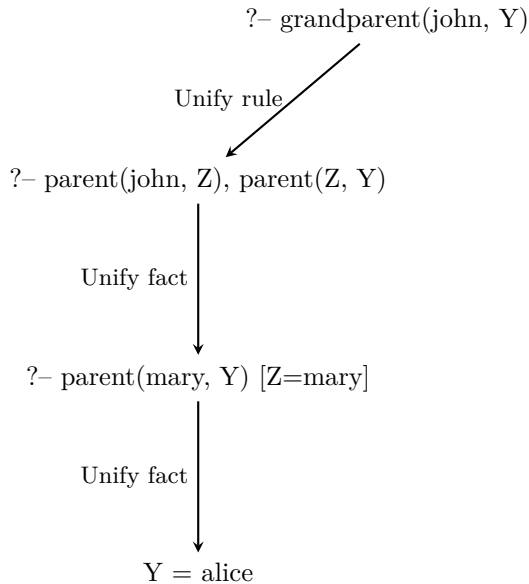
Equation of Unification: Two terms T_1 and T_2 unify if there exists a substitution θ such that:

$$T_1\theta = T_2\theta$$

Example: The query `?- foo(A, B) = foo(1, 2).` results in the substitution $\theta = \{A/1, B/2\}$.

4.2.2 Backtracking and the SLD Tree

Prolog uses a **Depth-First Search (DFS)** strategy to explore the knowledge base. If it reaches a “dead end” (a goal that cannot be proved), it “backtracks” to the last choice point and attempts an alternative rule. This process can be visualized as an **SLD Tree** (Selective Linear Definite clause resolution).



4.3 List Processing and Recursion

Lists are the primary data structure in Prolog. A list is either empty (`[]`) or consists of a **Head** (the first element) and a **Tail** (the rest of the list). The notation `[H|T]` is used to deconstruct a list.

Example: Counting Elements

```
count([], 0).
count([_|T], N) :- count(T, N1), N is N1 + 1.
```

This recursive rule states that the count of an empty list is 0, and the count of a non-empty list is 1 plus the count of its tail.

Mathematically, a list `[a, b, c]` is a nested structure:

$$.(a, .(b, .(c, [])))$$

where the dot (`.`) is the list constructor.

4.4 Controlling Search: The Cut Operator (!)

Prolog’s default backtracking behavior can sometimes lead to inefficiency or incorrect logic. The **Cut** operator (!) is used to “freeze” the current choices. Once a cut is encountered, Prolog commits to the current branch and will not backtrack past the cut for that specific goal.

Example: Mutually Exclusive Rules

```
perceive_hint(Secret, Guess, hot) :- Guess > Secret, !.
perceive_hint(Secret, Guess, cold) :- Guess < Secret, !.
perceive_hint(Secret, Guess, found) :- Guess == Secret.
```

Here, if `Guess > Secret` is true, the cut prevents Prolog from even attempting the `cold` or `found` rules, saving computation time and ensuring only one hint is returned.

4.5 Dynamic Knowledge Bases and Agent Memory

While traditional Prolog is static, intelligent agents often need to update their knowledge as they explore an environment. Prolog provides **dynamic predicates** for this purpose.

- `:- dynamic predicate/arity.` Declares that a predicate can be modified at runtime.
- `assertz(Fact).` Adds a new fact to the end of the KB.
- `retract(Fact).` Removes a fact from the KB.

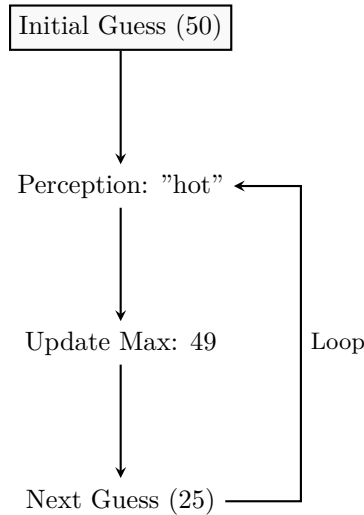
Example: Updating Agent Location

```
move(NewRoom) :-
    retract(location(agent, _)),
    assertz(location(agent, NewRoom)).
```

This simple rule allows an agent to “forget” its old location and “remember” a new one, effectively managing state within a declarative framework.

4.6 Case Study: The Hot or Cold Agent

Consider an agent tasked with finding a secret number within a range. The environment provides feedback: “hot” if the guess is too high and “cold” if it is too low. The agent uses a binary search strategy implemented in Prolog.



The agent logic relies on the `is` operator for arithmetic evaluation and `//` for integer division. The core of the search is a recursive `solve` predicate that updates boundaries until the `found` condition is met.

4.7 Integration: Prolog and Python

While Prolog excels at reasoning, other languages like Python are often preferred for data processing or user interfaces. The **PySwip** library allows for seamless integration, enabling Python programs to consult Prolog files and query the knowledge base as if it were a native data source. This hybrid approach—using Python for “muscles” and Prolog for the “brain”—is a powerful pattern for modern AI systems.

5 Rule-Based Systems and Rule Engines

As logic-based agents evolve from theoretical models into practical enterprise tools, the focus shifts from pure mathematical proofs to efficient, scalable automation. Rule Engines represent this transition, bridging the gap between formal logic—such as Prolog—and the demands of modern software architectures. By separating business logic from application code, rule engines enable organizations to manage complex decision-making processes with agility and transparency.

5.1 From Logic to Automation

The journey toward rule-based systems begins with the limitations of Boolean logic. While Boolean logic is sufficient for simple true/false assertions, it lacks the vocabulary to describe relationships between objects or the properties of groups. **First-Order Logic (FOL)** addresses this by introducing objects (nouns), predicates (properties or relations), and quantifiers ($\forall, \exists$).

To ensure computational efficiency, practical systems often restrict FOL to **Horn Clauses**. A Horn clause is a specific logical structure containing at most one positive literal. Mathematically, it is represented as an implication:

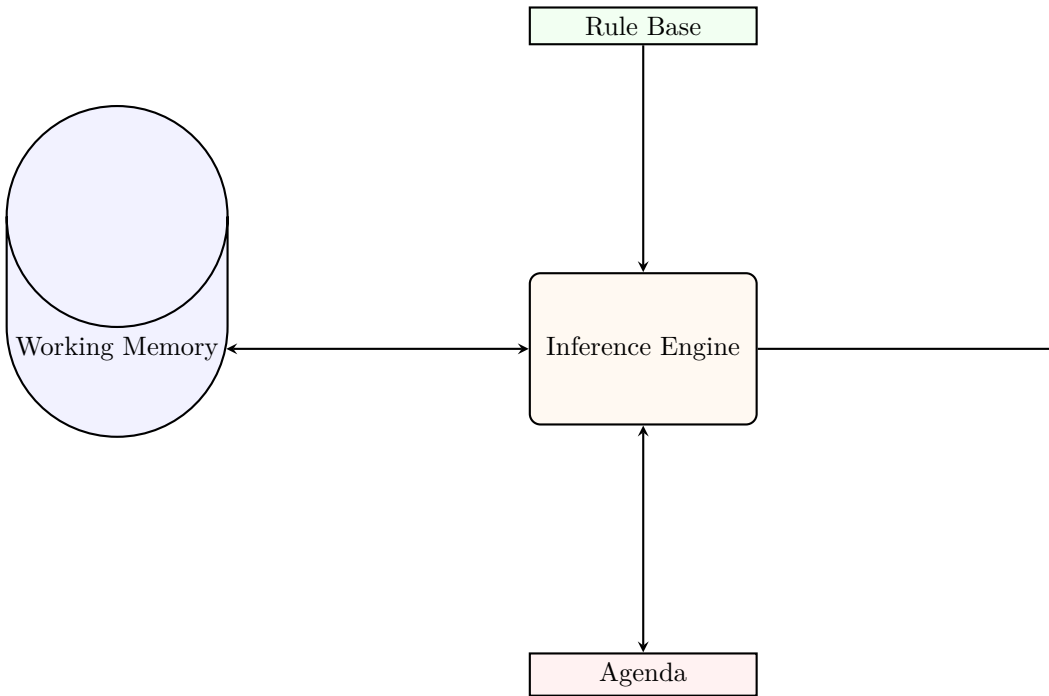
$$P_1 \wedge P_2 \wedge \cdots \wedge P_n \Rightarrow Q$$

This structure is the foundation of logic programming and allows for highly efficient resolution algorithms, as the engine only needs to find a path to a single conclusion Q .

5.2 The Architecture of a Rule Engine

A Rule Engine, or Business Rules Management System (BRMS), is a specialized piece of software designed to execute “if-then” logic over a set of facts.

Unlike a monolithic program where logic is hard-coded, a rule engine maintains a clear separation between the data and the reasoning.



1. **Rule Base:** A repository of declarative “If-Then” rules.
2. **Working Memory:** A dynamic storage area for transient data (facts), such as an incoming sensor reading or a customer’s credit score.
3. **Inference Engine:** The core processing unit that matches facts against rules.
4. **Agenda:** A conflict resolution manager that determines the execution order when multiple rules are triggered.

5.3 The Decision Cycle: Match-Resolve-Act

The operation of a rule engine is cyclic, driven by changes in the Working Memory. This process is generally known as **Forward Chaining**, as it starts with new data and moves forward to find applicable conclusions.

5.3.1 Step 1: Match

The engine continuously compares the facts in the Working Memory against the conditions of every rule in the Rule Base. This pattern matching is often optimized using the **Rete Algorithm**, which creates a network of nodes to minimize redundant evaluations.

5.3.2 Step 2: Resolve (Conflict Resolution)

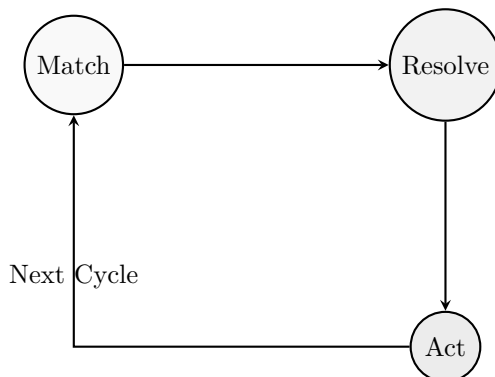
If multiple rules match the same set of facts, they are added to the **Agenda**. The engine must then decide which rule to fire based on a **Hit Policy**. Common policies include: * **First Match**: Execute only the first rule that meets the criteria. * **Priority (Salience)**: Execute rules in order of an assigned numerical weight. * **Collect**: Execute all matching rules and return a cumulative result.

Mathematically, the resolution can be seen as selecting an action a from the set of triggered rules R :

$$a = \text{Resolve}(\{r \in R \mid \text{Match}(r, \text{Facts})\})$$

5.3.3 Step 3: Act

The engine executes the consequence of the selected rule. This action can involve returning a result, triggering an external API, or even asserting new facts into the Working Memory, which may trigger further rules in a cascading effect.



5.4 Real-World Applications

Rule engines excel in domains where logic is complex, changes frequently, and must be auditable.

- **Finance and Insurance:** Automated loan approvals evaluate credit history against bank policies. Fraud detection systems scan transactions for malicious patterns in real-time.
- **Healthcare:** Expert systems provide diagnostic recommendations by comparing patient symptoms against vast medical databases. Treatment protocols ensure new prescriptions do not conflict with known allergies.
- **IoT and Automation:** Managing smart home environments where HVAC systems or alarms are triggered based on environmental thresholds (e.g., `if humidity > 65% then turn_on_dehumidifier`).

Example: Automated Loan Approval Consider a bank's rule base:
1. Rule 1: If `CreditScore > 700` and `Income > 50000` then `Status = Approved`.
2. Rule 2: If `CreditScore < 600` then `Status = Rejected`.
When a customer's data (facts) enters the Working Memory, the engine matches these against the Rule Base. If both rules were to somehow match (conflict), the Priority policy might give precedence to the `Rejected` status for safety.

5.5 Modern Implementations: GoRules and JDM

Modern rule engines like **GoRules** have evolved to be cloud-native and user-friendly. They replace abstract code with visual **Decision Tables**—spreadsheets where columns represent inputs and outputs. Under the hood, these are serialized into a **JSON Decision Model (JDM)**, allowing business analysts to update logic without developer intervention.

The advantage of this approach is threefold: 1. **Agility:** Logic can be updated instantly without a full software redeploy. 2. **Transparency:** It is easy to audit a decision table to see *why* a customer was rejected. 3. **Separation of Concerns:** Developers focus on infrastructure while business experts focus on the rules.

5.6 Conclusion

Rule Engines provide a robust bridge between the symbolic reasoning of logic agents and the practical needs of automated systems. By formalizing the Match-Resolve-Act cycle and providing tools for declarative decision-making, they enable the creation of intelligent systems that are both powerful and maintainable. While they introduce architectural overhead, their ability to provide auditability and business agility makes them indispensable in modern enterprise AI.

6 Bayesian Agents and Probabilistic Reasoning

While logic-based agents provide a powerful framework for reasoning in structured environments, they often struggle with the messy, uncertain reality of the physical world. In a purely logical system, an agent requires an exhaustive list of preconditions to guarantee an action—a challenge known as the **Qualification Problem**. Furthermore, the **Laziness Problem** highlights the impracticality of accounting for every possible exception. To thrive in environments where knowledge is incomplete and observations are noisy, agents must shift from the binary certainty of logic to the numerical “degrees of belief” provided by **Probabilistic Reasoning**.

6.1 From Certainty to Uncertainty

In logic, a sentence is either True, False, or Unknown. In contrast, a probabilistic agent assigns a probability between 0 and 1 to every state, representing its subjective uncertainty. This shift allows the agent to handle: * **Theoretical Ignorance**: Not knowing all the rules governing the environment. * **Practical Ignorance**: Not having enough sensors to observe the full state of the world.

The core of this approach is the **Bayesian** perspective, where probability is seen as an agent’s belief that is updated as new evidence arrives. This is fundamentally different from the **Frequentist** view, which defines probability as the long-run frequency of an event.

6.1.1 Random Variables and Distributions

We represent aspects of the world using **Random Variables** (X). Each variable has a **Sample Space** (Ω)—the set of all possible outcomes. A **Probability Distribution** assigns a value to each outcome such that the sum of

all probabilities is exactly 1.

$$\sum_{x \in \Omega} P(X = x) = 1$$

The **Law of Total Probability** allows an agent to determine the probability of an event A by considering all the mutually exclusive ways it could occur:

$$P(A) = \sum_i P(A|B_i)P(B_i)$$

6.2 The Heart of Bayesian Inference: Bayes' Rule

The most critical tool for a probabilistic agent is **Bayes' Rule**. It provides a mathematical mechanism for the agent to update its “World Model” based on new sensor data. It relates the **Prior** (what was known before) and the **Likelihood** (how well the evidence explains a cause) to the **Posterior** (the updated belief).

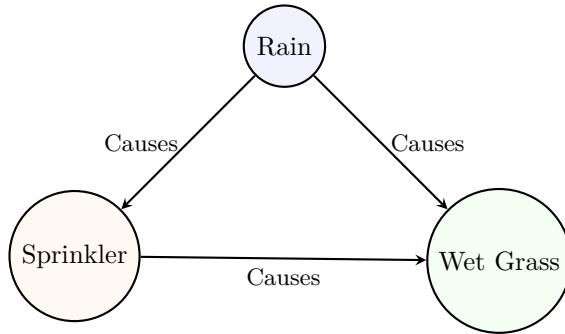
Equation of Bayes' Rule:

$$P(Cause|Effect) = \frac{P(Effect|Cause)P(Cause)}{P(Effect)}$$

Example: The Noisy Sensor Consider an agent that must decide if a person is behind a door based on a noisy motion sensor. * Prior: $P(Person) = 0.3$. * Likelihood (True Positive): $P(Sensor|Person) = 0.9$. * False Positive: $P(Sensor|\neg Person) = 0.2$. If the sensor triggers, the agent applies Bayes' rule to find that the probability of a person being there has increased from 30% to approximately 66%, allowing it to make a more informed decision to open the door.

6.3 Bayesian Networks

A **Bayesian Network** is a graphical model that represents the probabilistic relationships between variables. It consists of a **Directed Acyclic Graph (DAG)** where nodes are random variables and edges represent direct causal influences.



The power of a Bayesian Network lies in its ability to factor the **Joint Probability Distribution** into smaller, local conditional probability tables (CPTs). This reduces the complexity of the problem from exponential to linear in many cases:

$$P(x_1, \dots, x_n) = \prod_{i=1}^n P(x_i | \text{parents}(x_i))$$

6.4 Modeling Decisions over Time: Markov Processes and MDPs

Agents often operate in environments where decisions have long-term consequences. To prevent a “memory explosion,” we often assume the **Markov Property**: the future depends only on the current state, not the past.

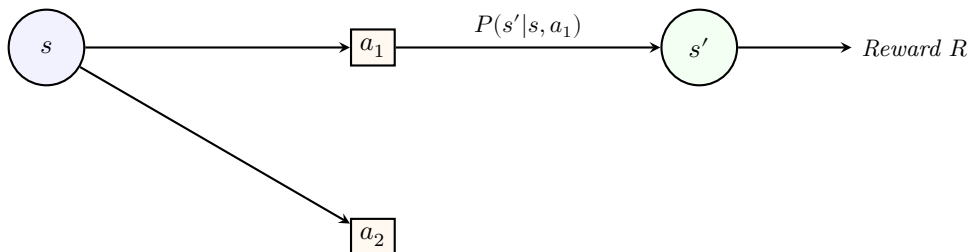
6.4.1 Markov Decision Processes (MDP)

An MDP extends this idea by adding actions and rewards. It is defined by: 1. **States (S)**: The set of possible situations. 2. **Actions (A)**: What the agent can do. 3. **Transition Model** $P(s'|s, a)$: The probability of reaching state s' from s given action a . 4. **Reward Function** $R(s, a, s')$: The immediate payoff for a transition.

The goal of the agent is to find an **Optimal Policy** (π^*) that maximizes the **Expected Utility**. Because future rewards are uncertain, we use a **Discount Factor** ($\gamma \in [0, 1]$) to value immediate rewards more than distant ones.

The Bellman Equation: The value of a state $V(s)$ is the maximum expected return the agent can achieve from that state:

$$V(s) = \max_a \sum_{s'} P(s'|s, a) [R(s, a, s') + \gamma V(s')]$$



6.5 Training and Implementation

Building a Bayesian agent involves learning both the structure of the network and the values in the CPTs. * **Maximum Likelihood Estimation (MLE):** Counting observations to fill the tables. * **Monte Carlo Methods:** Running thousands of random simulations (trajectories) to estimate the value of states when the transition model is unknown.

Modern tools like **pgmpy** (for Bayesian Networks) and **Gymnasium** (for MDPs and Reinforcement Learning) allow developers to implement these complex mathematical models efficiently. By separating the “Prior” knowledge from the “Evidence” gathered by sensors, Bayesian agents can make rational decisions in an inherently uncertain world.